



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO
Facultad de Ciencias

Diplomado Introducción a la Ciencia de Datos

Módulo 5

Profesores: Alan Riva Palacio Cohen y
Jorge Ignacio González Cázares

Informe de Predicción de Rotación de Empleados

Equipo 22

Elaboró:

- Betancourt Peralta Diego
- Canul Hernández Erick Iván
- Gutiérrez Gutiérrez José Omar
- Hernández Pérez Maximiliano
- Torres Vargas Brenda Poulette

Índice

1. Resumen Ejecutivo	3
2. Planteamiento del Problema	3
3. Exploración de Datos	4
4. Preparación de Datos	5
5. Modelos Base y Regularización	5
5.1. Modelos Base: Regresión Logística con Regularización (Lasso y Ridge)	5
5.2. Metodología de Optimización y Selección de Métricas en Grid Search	6
5.3. Interpretación de coeficientes y efecto de las penalizaciones Lasso y Ridge.	7
6. Modelos de Ensamble y Redes	8
6.1. Random Forest	8
6.2. AdaBoost	9
6.3. XGBoost.	10
6.4. Perceptrón Multicapa (MLP)	12
7. Evaluación Multimodelo: Rendimiento y Explicabilidad	12
7.1. Contraste de Rendimiento Predictivo	12
7.2. Análisis de la Explicabilidad: Lasso vs. XGBoost	13
8. Conclusiones	14

Predicción de Rotación de Empleados

1. Resumen Ejecutivo

La salida voluntaria de empleados representa un desafío para las organizaciones, ya que genera costos adicionales de reemplazo, capacitación e integración de nuevo talento, además de afectar la continuidad operativa y la productividad del equipo.

De modo que desarrollar modelos predictivos capaces de identificar anticipadamente a los colaboradores con mayor riesgo de abandono resulta de interés para orientar estrategias de retención y optimizar la gestión del talento humano.

En este sentido, utilizando el conjunto de datos **IBM HR Analytics Employee Attrition** de Kaggle, se desarrollaron y compararon modelos de regresión, ensamble y redes. Tras evaluar su desempeño, *XGBoost* resultó la alternativa más adecuada para apoyar la identificación temprana de empleados con riesgo de rotación y contribuir al diseño de estrategias preventivas de retención. Este modelo logró identificar correctamente del conjunto de prueba al 74 % de empleados que abandonaron la compañía y alcanzó, además, una precisión del 60 %.

2. Planteamiento del Problema

Como la rotación voluntaria de empleados representa un desafío para las organizaciones que se materializa en costos asociados al reclutamiento y selección de nuevo personal, inversiones en capacitación, reducción temporal de productividad y pérdida de conocimiento institucional, para las empresas resulta imperativo mitigar dichos riesgos a través de la identificación temprana de colaboradores con una mayor probabilidad de abandonar la compañía.

En este sentido, el presente análisis tiene como objetivo **desarrollar y comparar distintos modelos de aprendizaje supervisado para la predicción de la rotación voluntaria de empleados** a partir de variables demográficas, laborales, salariales y de satisfacción organizacional, contenidas en el conjunto de datos **IBM HR Analytics Employee Attrition (Kaggle)**.

Ahora bien, dado que se trata de un problema de **clasificación binaria**, se empleó la regresión logística como modelo base, complementándola con técnicas de regularización Ridge y Lasso, además de evaluar modelos de ensamble, tales como Random Forest, AdaBoost y XGBoost, así como un Perceptrón Multicapa (MLP), para comparar su desempeño predictivo en la identificación de empleados con riesgo de rotación.

El trabajo de este proyecto se realizó en un jupyter notebook, el cual puede encontrarse en el siguiente repositorio de GitHub: <https://github.com/EICanulH/Proyecto-Diplomado-Modulo-5>. En este también se encuentra el conjunto de datos en formato csv, así como las paqueterías del environment de Python con el que se trabajó en el archivo **requirements.txt**. Es importante aclarar que, dada la naturaleza colaborativa de este proyecto, utilizamos la herramienta de Google Colab para una mejor coordinación entre los autores de este documento, por lo que se recomienda ejecutarlo con esta herramienta para evitar problemas de compatibilidad con el código del notebook.

3. Exploración de Datos

En principio, observamos que el conjunto de datos de análisis, además de no presentar valores faltantes, cuenta con 1,470 observaciones y 35 variables, de las cuales 26 son numéricas (`int64`) y 9 categóricas (`object`).

Asimismo, se identificó que la variable objetivo, **Attrition**, corresponde a una variable categórica binaria; no obstante, su distribución evidenció un desbalance de clases considerable, ya que el 83.9% de los empleados permaneció en la compañía, mientras que únicamente el 16.1% presentó rotación. En consecuencia, la métrica de *accuracy* resulta insuficiente para evaluar el desempeño de los modelos, por lo que será necesario considerar métricas que reflejen de manera más adecuada la capacidad de identificar la clase minoritaria.

Finalmente, se detectó alta multicolinealidad, particularmente en las variables relacionadas con la antigüedad y salario del trabajador.

A continuación, se presenta una breve descripción de las variables que conforman el conjunto de datos original:

- **Age**: Edad del empleado.
- **Attrition**: Rotación laboral o deserción (Variable Objetivo: Sí / No).
- **BusinessTravel**: Frecuencia de viajes de negocio del empleado.
- **DailyRate**: Tarifa diaria o salario equivalente por día de trabajo.
- **Department**: Departamento de la empresa al que pertenece el empleado.
- **DistanceFromHome**: Distancia en kilómetros desde el hogar hasta el centro laboral.
- **Education**: Nivel educativo alcanzado por el empleado.
- **EducationField**: Campo de estudio o carrera profesional del empleado.
- **EmployeeCount**: Conteo de empleados (Constante con valor 1).
- **EmployeeNumber**: Identificador único o número de nómina del empleado.
- **EnvironmentSatisfaction**: Nivel de satisfacción con el entorno o clima laboral.
- **Gender**: Género del empleado.
- **HourlyRate**: Tarifa o costo por hora de trabajo.
- **JobInvolvement**: Grado de involucramiento, compromiso y centralidad del trabajo para el empleado.
- **JobLevel**: Nivel jerárquico o rango del puesto dentro de la organización.
- **JobRole**: Cargo o rol específico que desempeña el empleado.
- **JobSatisfaction**: Nivel de satisfacción intrínseca con las tareas de su puesto.
- **MaritalStatus**: Estado civil del empleado.
- **MonthlyIncome**: Ingreso o salario bruto mensual bruto.
- **MonthlyRate**: Tarifa mensual valorada de asignación técnica.
- **NumCompaniesWorked**: Número de empresas previas en las que ha laborado el empleado.
- **Over18**: Indicador de si el empleado es mayor de 18 años (Constante con valor 'Y').
- **OverTime**: Indicador de si el empleado realiza horas extra habitualmente (Sí / No).
- **PercentSalaryHike**: Porcentaje de aumento salarial recibido en el último ciclo.

- **PerformanceRating**: Calificación formal del rendimiento u evaluación del desempeño.
- **RelationshipSatisfaction**: Nivel de satisfacción con las relaciones interpersonales en el entorno laboral.
- **StandardHours**: Horas estándar de trabajo establecidas por contrato (Constante con valor 80).
- **StockOptionLevel**: Nivel de opciones de compra de acciones de la empresa otorgadas al empleado.
- **TotalWorkingYears**: Años totales de experiencia laboral acumulada a lo largo de su carrera.
- **TrainingTimesLastYear**: Cantidad de capacitaciones o entrenamientos tomados el año anterior.
- **WorkLifeBalance**: Percepción del equilibrio entre la vida laboral y la vida personal.
- **YearsAtCompany**: Años de antigüedad acumulados específicamente en la empresa actual.
- **YearsInCurrentRole**: Años transcurridos desempeñando el cargo o puesto actual.
- **YearsSinceLastPromotion**: Años transcurridos desde el último ascenso formal del empleado.
- **YearsWithCurrManager**: Años acumulados trabajando bajo la dirección del supervisor actual.

4. Preparación de Datos

Durante el proceso de selección de variables, se identificaron y eliminaron cuatro variables sin valor predictivo. En particular, **EmployeeCount**, **StandardHours** (numéricas) y **Over18** (categórica). Se descartaron por ser constantes, es decir, por contener exactamente el mismo valor para cada una de ellas dentro de todos los registros del conjunto de datos. Por otro lado, se eliminó **EmployeeNumber** (numérica) al tratarse de un identificador único que no tiene poder predictivo.

Además, las variables categóricas binarias **Attrition**, **Gender** y **OverTime** fueron transformadas a formato numérico mediante codificación binaria, de modo que el conjunto final de datos quedó conformado por 26 variables numéricas y 5 categóricas.

Previo a cualquier fase de transformación avanzada o ajuste de hiperparámetros, se procedió a realizar la división del conjunto de datos en subconjuntos de entrenamiento y prueba. Debido al desbalanceo crítico inherente al fenómeno de rotación, se implementó un muestreo estratificado enfocado en la variable objetivo (**Attrition**). Esta estrategia garantiza que ambos subconjuntos conserven idénticas proporciones de la clase minoritaria, blindando la validez estadística de las evaluaciones posteriores y previniendo el sesgo por fuga de datos (*data leakage*).

Posteriormente, con el fin de garantizar la compatibilidad de los atributos cualitativos con los algoritmos de aprendizaje supervisado considerados en el estudio, se aplicó la técnica de *one-hot encoding* a aquellas variables nominales con más de dos categorías.

Finalmente, dado que múltiples variables numéricas presentaban escalas considerablemente heterogéneas, se implementó una estandarización mediante **StandardScaler**. Este paso se aplicó de manera exclusiva previo al ajuste de los modelos altamente sensibles a la magnitud de las variables, particularmente la regresión logística regularizada (Ridge y Lasso) y el Perceptrón Multicapa (MLP).

5. Modelos Base y Regularización

5.1. Modelos Base: Regresión Logística con Regularización (Lasso y Ridge)

Como punto de partida (*baseline*), se seleccionaron dos variantes de Regresión Logística Regularizada: la variante Lasso, que incorpora una penalización L_1 , y la variante Ridge, que añade una penalización L_2 . Con el fin de sintonizar el balance óptimo entre el sesgo y la varianza, se implementó una búsqueda

exhaustiva de hiperparámetros (*GridSearchCV*) orientada a optimizar la fuerza de regularización de ambas normas.

Debido al desbalanceo nativo del problema, en ambos modelos se activó el parámetro `class_weight='balanced'`, el cual modifica la función de pérdida asignando pesos inversamente proporcionales a las frecuencias de las clases, forzando al optimizador a prestar mayor atención a la clase minoritaria (`Attrition = Yes`) en combinación con el efecto restrictivo de dichas penalizaciones.

- **Regresión Logística Lasso (Penalización L_1):** Este modelo introduce una penalización basada en el valor absoluto de los coeficientes $\lambda \sum_{j=1}^p |\beta_j|$. Se configuró utilizando el optimizador 'liblinear' para permitir la optimización de la norma L_1 . Su propósito principal en este estudio es actuar como un mecanismo de selección intrínseca de variables; al forzar a cero los coeficientes de los atributos redundantes o irrelevantes, simplifica la complejidad del modelo y mejora la interpretabilidad para el negocio.
- **Regresión Logística Ridge (Penalización L_2):** A diferencia de Lasso, la variante Ridge aplica una penalización basada en el cuadrado de los coeficientes del modelo $\lambda \sum_{j=1}^p \beta_j^2$. Se implementó utilizando el optimizador por defecto 'lbfgs' (*Limited-memory Broyden-Fletcher-Goldfarb-Shanno*), configurando un límite estricto de `max_iter=1000` para garantizar la convergencia matemática del gradiente. La inclusión de esta arquitectura es fundamental debido a la alta multicolinealidad detectada durante el análisis exploratorio (particularmente en las métricas de antigüedad y salario); la penalización L_2 encoge los coeficientes correlacionados de manera uniforme sin eliminarlos, estabilizando las predicciones ante pequeñas variaciones en los datos de entrada.

5.2. Metodología de Optimización y Selección de Métricas en Grid Search

Para ambas arquitecturas lineales, la cuadrícula de búsqueda (*GridSearchCV*) evaluó el hiperparámetro de escala de regularización inversa `model__C` a través de nueve órdenes de magnitud: $C \in \{0.001, 0.01, 0.1, 0.25, 0.5, 0.75, 1.0, 10.0, 100.0\}$. Valores más bajos de C incrementan la fuerza de la regularización para combatir el sobreajuste (*overfitting*), mientras que valores altos permiten al modelo ajustarse con mayor libertad a los datos de entrenamiento.

Para la optimización de los hiperparámetros de los algoritmos, se implementó una estrategia de validación cruzada estratificada de 5 pliegues (*5-fold Stratified K-Fold*), asegurando la reproducibilidad mediante una semilla aleatoria fija (`random_state=123`). El criterio de selección del mejor modelo dentro de la cuadrícula de búsqueda (*GridSearchCV*) requirió un análisis crítico debido al severo desbalanceo de clases identificado en la variable objetivo (`Attrition: Yes  $\approx$  16 %, No  $\approx$  84 %`).

En este contexto, se descartó de forma categórica el uso de la métrica de exactitud global (*Accuracy*), dado que un clasificador ingenuo que prediga sistemáticamente la permanencia de todos los empleados alcanzaría un 84 % de aciertos sin aportar valor predictivo real.

Por otro lado, al representar la media armónica entre la precisión (*Precision*) y la sensibilidad (*Recall*), optimizar el modelo bajo la métrica *F1-score* garantiza el balance óptimo para el negocio: maximizar la captura de empleados en riesgo real de deserción (evitando falsos negativos) sin saturar al departamento de Recursos Humanos con falsas alarmas que deriven en costos innecesarios de retención (mitigando falsos positivos).

Por consiguiente, se determinó configurar el parámetro `scoring="f1"` enfocado exclusivamente en la clase positiva (`Attrition = Yes`).

La selección de este criterio responde directamente al dilema técnico del compromiso entre precisión y sensibilidad (*Precision/Recall Trade-off*). Si se hubiese optado por optimizar mediante `recall`, el algoritmo habría priorizado la detección masiva de bajas a costa de inundar la organización con alertas falsas y costosas. Por el contrario, optimizar con `precision` habría vuelto al sistema ultra-conservador, alertando únicamente ante certezas absolutas y dejando escapar de forma masiva el talento en riesgo.

Tras ejecutar la búsqueda en cuadrícula (*GridSearchCV*) optimizada para maximizar el *F1-score* de la clase positiva, se evaluó el rendimiento de las dos variantes lineales regularizadas sobre el conjunto de datos de prueba independiente. Ambas arquitecturas lograron mitigar de manera eficiente el sesgo del desbalanceo de clases gracias a la incorporación del ajuste de pesos en la función de pérdida. Los resultados globales de rendimiento se consolidan en la siguiente tabla.

Regularización	C	Precision	Recall	F1-Score	TP	FN	FP
Lasso (L_1)	0.5	0.36	0.74	0.49	35	12	62
Ridge (L_2)	0.25	0.36	0.74	0.49	35	12	62

Tabla 1: Métricas de rendimiento de los modelos base de Regresión Logística.

Desde el punto de vista del negocio y los objetivos de Recursos Humanos, ambos modelos logran una sensibilidad (*Recall*) destacable del 74 %, lo que significa que el sistema es capaz de detectar correctamente a 35 de los 47 empleados que realmente causarán baja en la empresa ($TP = 35$, $FN = 12$). No obstante, este rendimiento preventivo se penaliza con una precisión moderada del 36 %, generando 62 falsos positivos (FP). Esta cantidad de falsas alarmas establece un punto de partida que buscaremos reducir en las siguientes secciones del reporte, poniendo a prueba a modelos más avanzados y complejos como Random Forest, AdaBoost, XGBoost y el Perceptrón Multicapa (MLP).

5.3. Interpretación de coeficientes y efecto de las penalizaciones Lasso y Ridge.

A partir de los valores obtenidos, las variables con mayor impacto en el modelo se agrupan en cuatro categorías estratégicas para el departamento de Recursos Humanos:

- Los viajes de negocios frecuentes (**BusinessTravel_Travel_Frequently** $\approx +1.01$) representan el factor de riesgo más alto, seguidos de cerca por los empleados solteros (**MaritalStatus_Single** $\approx +0.92$). Esto indica que la alta movilidad y la falta de balance vida-trabajo aceleran la decisión de abandonar la empresa.
- Los puestos de Técnico de Laboratorio ($\approx +0.79$) y Representante de Ventas ($\approx +0.58$) muestran una clara tendencia a la renuncia. Como contraparte, pertenecer al departamento de Investigación y Desarrollo (**Department_Research & Development**) actúa como un fuerte factor de retención, recibiendo un peso negativo de -0.8429 en Lasso y -0.2974 en Ridge.
- El estancamiento laboral incrementa el riesgo de salida, lo cual se refleja en los coeficientes positivos de los años desde el último ascenso (**YearsSinceLastPromotion** $\approx +0.55$) y los años totales en la empresa (**YearsAtCompany** $\approx +0.46$). Sin embargo, tener estabilidad con el jefe actual (**YearsWithCurrManager** ≈ -0.48) y permanencia en el puesto actual (**YearsInCurrentRole** ≈ -0.50) mitigan este riesgo, demostrando que ambas situaciones son retenedores claves.
- Tanto el salario bruto (**MonthlyIncome** ≈ -0.28) como las opciones de acciones (**StockOptionLevel** ≈ -0.22) ayudan a disminuir la desertión. Finalmente, una buena satisfacción laboral (**JobSatisfaction** ≈ -0.40) y un clima laboral positivo (**EnvironmentSatisfaction** ≈ -0.38) reducen drásticamente las probabilidades de rotación.

Efecto de las penalizaciones Lasso y Ridge

- **Efecto de la Penalización Lasso (L_1):** La optimización seleccionó un valor óptimo de $C = 0.50$ a través de la validación cruzada. Desde la perspectiva analítica, esta regularización cumplió de forma efectiva su función de simplificación estructural sobre el espacio expandido de 44 variables transformadas, logrando reducir a cero los coeficientes de 8 atributos redundantes o irrelevantes (tales como **HourlyRate**, **Department_Sales**, **EducationField_Marketing**, **EducationField_Other**, y roles como **Human Resources**, **Research Director**, **Research Scientist** y **Sales Executive**).
- **Efecto de la Penalización Ridge (L_2):** Con un valor óptimo de $C = 0.25$, esta opción mantuvo activas las 44 variables del dataset sin eliminar ninguna. A diferencia de Lasso, que borró el puesto **JobRole_Sales Executive**, Ridge lo conservó con un peso de $+0.1529$. En el caso de variables muy parecidas, como los años con el jefe actual (**YearsWithCurrManager**), ambos modelos coincidieron en darle un impacto negativo similar (-0.4808 en Lasso y -0.4987 en Ridge). Esto demuestra que Ridge no descarta información, sino que reparte el peso de forma más equilibrada para que el modelo sea más estable.

6. Modelos de Ensamble y Redes

6.1. Random Forest

Para tunear los hiperparámetros de `RandomForestClassifier` se utilizó `GridSearchCV`. El mejor conjunto de hiperparámetros se decide a través de un puntaje, el cual se pasa al parámetro `scoring` de `GridSearchCV`. De manera predeterminada, este puntaje es *accuracy* ('`accuracy`'); sin embargo, como las clases no están balanceadas, se obtuvieron mejores resultados con puntajes como F1 ('`f1`'), *weighted* F1 ('`f1_weighted`') y *weighted recall* ('`recall_weighted`'). Se hizo pruebas con *recall* ('`recall`'), pero el algoritmo arrojaba conjuntos de parámetros de bosques que clasificaban como 1 a casi todas las observaciones del conjunto de prueba, pues, en efecto, los bosques entrenados con estos parámetros obtendrían valores de *recall* cercanos a 1 en cada iteración del *grid search*.

También se realizaron pruebas utilizando como puntajes el área bajo la curva ROC ('`roc_auc`') y el coeficiente de correlación de Matthews. Para este último puntaje debía pasarse al argumento `scoring` la función `make_scorer()` de `sklearn.metrics`, evaluada en `matthews_corrcoef` del mismo módulo, es decir, `score=make_scorer(matthews_corrcoef)`.

Aunque estos dos puntajes son más adecuadas para medir el desempeño de modelos de clasificación con datos desbalanceados, los modelos obtenidos con estos puntajes eran generalmente peores que los obtenidos con F1, *weighted* F1 y *weighted recall*. Aun así, algunos modelos obtenidos con el CCM eran un poco mejores en algunos aspectos.

También hicimos pruebas pasando métricas definidas por nosotros a la función `make_scorer()`. Con una de estas métrica se obtuvieron buenos valores de *accuracy* y *precision*, pero pobres de *recall*. Veamos cómo definimos el puntaje. Sean *TP*, *FN* y *FP* el número de positivos verdaderos, falsos negativos y falsos positivos obtenidos en una iteración del *grid search*. El puntaje se calcula como

$$\frac{TP}{FN + 6 \cdot FP}$$

Entre mayor fuera el número de positivos clasificados correctamente y menor fueran el número de datos clasificados incorrectamente, tanto falsos positivos como falsos negativos, el puntaje aumentaría. Notemos que *FP* se multiplica por 6 para penalizar todavía más los falsos positivos, con lo que mejoramos *precision*. Además, como el número de negativos es mucho mayor, tener menos falsos positivos mejora de manera general el *accuracy*. Notemos que con esta métrica el *recall* no mejora casi nada; aun así, los resultados no fueron los peores obtenidos.

El algoritmo de árbol aleatorio le da el mismo peso de 1 a cada clase a la hora de dividir cada nodo, lo que ocasiona que los valores de *recall* y *precision* no sean buenos con el conjunto de prueba. Es por esto que debemos utilizar el parámetro `class_weight` de `RandomForestClassifier`, al cual se le pueden asignar los argumentos '`balanced`', '`balanced_subsample`' o diccionarios en los que se asigna a cada categoría un peso, por ejemplo {0: 1, 1: 8}. Al utilizar '`balanced`', a cada valor de la variable objetivo se le asigna un peso inversamente proporcional a su frecuencia. A continuación se explica cómo se calcula. Sea *N* el número de observaciones en el conjunto utilizado para entrenar el bosque aleatorio, *c* el número de clases y *n_i* el número de observaciones de la clase *i*. A cada clase *i* se le asigna el siguiente peso:

$$\frac{N}{c \cdot n_i}.$$

La estrategia de validación cruzada que utilizamos en el *grid search*, `StratifiedKFold`, hace que en cada iteración haya una proporción de clases muy similar a la proporción del conjunto de entrenamiento completo `y_train`. Así mismo, `y_train` conserva la proporción de clases de `y`, el vector de observaciones completo, debido a que se utilizó el parámetro `stratify` con argumento `y` de la función `train_test_split()`. De este modo, el peso asignado a cada clase en cada iteración del *grid search* es similar a la que se le asignaría a cada clase si entrenáramos el bosque con `y`. Así pues, el peso de la clase 0 en cada iteración del *grid search* sería aproximadamente

$$\frac{1470}{2 \cdot 1233} \approx 0.5961$$

y la de la clase 1,

$$\frac{1470}{2 \cdot 237} \approx 3.1013.$$

Con el argumento '`balanced_subsample`', los pesos de cada clase se calculan individualmente para cada árbol, utilizando para ello el conjunto bootstrap que les tocó.

También hicimos pruebas asignando manualmente un peso a cada clase, pasando al argumento `class_weight` diccionarios como `{0: 1, 1: 6}`, `{0: 1, 1: 7}`, `{0: 1, 1: 8}` para darle todavía más peso a la clase minoritaria que la que se le da al pasar los argumentos '`balanced`' y '`balanced_subsample`'.

Para limitar el tamaño de los árboles podemos utilizar los parámetros `max_depth` o `min_samples_leaf`. Al primer parámetro se le puede asignar un entero que determina la profundidad máxima de cada árbol; al segundo se le puede asignar un entero que determina el número mínimo de instancias que debe haber en cada hoja, o bien, un flotante mayor a 0.0 y menor o igual a 1.0, que representa el porcentaje mínimo de instancias que debe haber en cada hoja. Decidimos tunear el parámetro `min_samples_leaf` pasándole porcentajes en vez de `max_depth`.

Para agregar aleatoriedad a cada árbol y hacerlos más independientes entre sí, también utilizamos el parámetro `max_features` para que cada uno de estos se entrenara con un subconjunto aleatorio de todas las *features*. Este puede ser un entero, que representa el número de features, seleccionadas aleatoriamente, con el que se entrena cada árbol o un porcentaje de todas las *features*. Se decidió tunear este hiperparámetro con el porcentaje, el cual puede ser cualquier valor mayor a 0.0 y menor o igual a 1.0.

El número de árboles por bosque (parámetro `n_estimators`) se fue variando en función de la cantidad de conjuntos de hiperparámetros que se iban a utilizar en una ejecución de `GridSearchCV` para reducir el tiempo de cómputo. Como criterio de división de los nodos (parámetro `criterion`) utilizamos el predeterminado de impureza de Gini ('`gini`'). Para el resto de parámetros también se utilizaron sus valores predeterminados.

Después de muchas búsquedas con `GridSearchCV`, el bosque con el mejor desempeño en el conjunto de prueba fue uno con los siguientes parámetros y argumentos:

- `n_estimators=1000`
- `max_features=0.4875`
- `min_samples_leaf=0.01325`
- `class_weight={0: 1, 1: 8}`
- Valores predeterminados para el resto de parámetros

Se obtuvo este bosque con un *grid search* utilizando el puntaje de *weighted* F1. En la tabla 2 se presentan las métricas de desempeño de este bosque con el conjunto de prueba:

Precisión (1)	<i>Recall</i> (1)	F1 (1)	TP	FN	FP
0.45	0.77	0.57	36	11	44

Tabla 2: Resultados del mejor bosque aleatorio encontrado con *grid search*.

6.2. AdaBoost

Para este modelo se implementó una versión manual de AdaBoost (`AdaBoostManualV3`) mediante una clase compatible con `BaseEstimator/ClassifierMixin`, integrada en un pipeline junto con el mismo preprocesamiento (`ColumnTransformer`) utilizado por el resto de los modelos del proyecto, garantizando que las transformaciones se ajusten únicamente con los datos de entrenamiento. A diferencia de la versión clásica de AdaBoost, donde los árboles débiles son *stumps* de profundidad fija igual a 1, se volvió `max_depth` un hiperparámetro buscable. Asimismo, dado que el parámetro `class_weight="balanced"` entra en conflicto con el reponderado interno que AdaBoost ya realiza sobre las muestras mal clasificadas en cada iteración (degradando el desempeño al duplicar ese mecanismo), el desbalance de clases no se inyectó en el entrenamiento, sino que se controló *a posteriori*, mediante un umbral de decisión ajustable sobre la probabilidad estimada:

$$\hat{y} = \mathbb{1} [P(\text{Attrition} = \text{Sí} \mid x) \geq \text{threshold}],$$

donde un **threshold** menor a 0.5 reduce la evidencia requerida para predecir “Se va”, incrementando el recall de la clase minoritaria a costa de precisión. Este mecanismo cumple, dentro de AdaBoost, un rol análogo al de **scale_pos_weight** en XGBoost: ambos son palancas para sesgar el modelo hacia la detección de la clase de interés, pero operando en etapas distintas del pipeline (entrenamiento vs. decisión).

La búsqueda de hiperparámetros se realizó mediante **GridSearchCV** con validación cruzada estratificada de cinco particiones (**StratifiedKFold**, **random_state=123**), preservando la proporción original de clases ($\sim 84\%/16\%$) en cada fold, de forma consistente con el resto de los modelos comparados en este trabajo. Se evaluaron dos estrategias de optimización sucesivas. La primera (V2) usó *recall* como métrica de selección del mejor modelo; sin embargo, optimizar exclusivamente sobre recall no penaliza de ningún modo la pérdida de precisión, lo cual sesgó la búsqueda hacia configuraciones extremas: los mejores hiperparámetros se ubicaron en el borde superior/inferior de la malla (**max_depth=3**, **n_estimators=150**, **threshold=0.3**), señal de que el espacio de búsqueda y la métrica no eran adecuados para capturar un balance razonable. Por ello, en la segunda iteración (V3) se amplió la malla y se reemplazó la métrica de optimización por el *F2-score* (Fbeta con $\beta = 2$):

$$F_{\beta} = (1 + \beta^2) \frac{P \cdot R}{\beta^2 P + R}, \quad \beta = 2,$$

que pondera el recall el doble que la precisión. Por lo que la métrica de selección del modelo debe favorecer explícitamente el recall sin renunciar por completo al control de falsos positivos, a diferencia de optimizar sobre recall puro.

Los resultados obtenidos para la clase positiva (“Se va”) con ambas configuraciones se muestran en la siguiente tabla:

Configuración	Precisión (1)	Recall (1)	F1 (1)	TP	FN	FP
Recall	0.62	0.21	0.32	10	37	6
Recall + threshold tuning	0.61	0.36	0.45	17	30	11
F2 ($\beta = 2$) + threshold tuning	0.49	0.57	0.53	27	20	28

Tabla 3: Comparación de resultados — AdaBoost Manual (V2 vs. V3)

Notemos que la configuración basada únicamente en recall (V2) resulta excesivamente conservadora: identifica correctamente solo al 36 % de los empleados en riesgo de abandono (17 de 47), dejando sin detectar a 30 de ellos, aunque mantiene un número bajo de falsas alarmas (11, equivalente al 4.5 % de los empleados que en realidad permanecieron). Dado que el costo de negocio asumido penaliza más fuertemente los falsos negativos que los falsos positivos, este comportamiento resulta inadecuado para el objetivo planteado. La configuración F2 (V3), en cambio, eleva la detección al 57 % (27 de 47), un incremento de 10 verdaderos positivos adicionales, a costa de 17 falsos positivos más (28 en total, 11.4 % de los empleados que permanecieron). Este incremento en falsas alarmas es aceptable bajo el supuesto de negocio, pues el costo de una intervención preventiva sobre un empleado que de cualquier forma habría permanecido es marginal frente al costo de no haber actuado sobre un empleado que efectivamente abandona la compañía. Por lo tanto, se seleccionó la configuración F2 + threshold tuning como modelo final de esta familia.

Así pues, la combinación óptima identificada por **GridSearchCV** fue la siguiente:

Hiperparámetro	n_estimators	max_depth	threshold
Valor óptimo	200	1	0.1

Tabla 4: Hiperparámetros del modelo AdaBoost Manual final (V3)

Cabe mencionar que cada malla evaluó 45 combinaciones de hiperparámetros, resultando en 225 ajustes por versión (450 modelos ajustados en total entre V2 y V3).

6.3. XGBoost.

Primeramente, el modelo fue implementado mediante un pipeline que integra el preprocesamiento y el algoritmo de clasificación en una sola estructura, garantizando que las transformaciones se ajusten utilizando únicamente los datos de entrenamiento y posteriormente se apliquen al conjunto de prueba.

Luego, para compensar el desbalance de clases en la posterior inicialización del clasificador `xgb_base`, se utilizó el parámetro `scale_pos_weight`, definido como:

$$\text{scale_pos_weight} = \frac{n_{No}}{n_{Yes}} = \frac{1,233}{237} \approx 5.2$$

Donde n_{No} representa el número de empleados que no abandonaron la compañía y n_{Yes} el número de empleados que sí abandonaron.

De modo que, durante el entrenamiento, los errores al clasificar empleados que abandonaron la empresa, recibieron un peso aproximadamente 5.2 veces mayor que los errores sobre empleados que no abandonaron, con el objetivo de identificar especialmente a los empleados en riesgo de abandono.

La búsqueda de hiperparámetros se realizó mediante `GridSearchCV` con validación cruzada estratificada de cinco particiones (`StratifiedKFold`), garantizando que cada partición conservara una distribución similar de empleados que abandonaron y no abandonaron la compañía, y con el propósito de identificar la configuración más adecuada para el problema, se evaluaron distintas combinaciones de métricas de optimización, Y el uso del parámetro `scale_pos_weight`. Los resultados obtenidos para la clase positiva se muestran en la siguiente tabla:

Configuración	Precisión (1)	Recall (1)	F1 (1)	TP	FN	FP
Recall	0.76	0.40	0.53	19	28	6
Recall + <code>scale_pos_weight</code>	0.43	0.79	0.55	37	10	50
f1 score + <code>scale_pos_weight</code>	0.53	0.62	0.57	29	18	26
f beta score con $\beta = 2$ + <code>scale_pos_weight</code>	0.50	0.74	0.60	35	12	35

Tabla 5: Comparación de Resultados de diferentes Estrategias de Optimización

Notemos que cuando no se emplea ninguna estrategia para compensar el desbalance de clases y la optimización se realiza con base en el *recall*, el modelo resulta excesivamente conservador, pues sólo identifica al 40 % de empleados en riesgo de abandono (19 de 47) y clasifica incorrectamente a 6 empleados que en realidad permanecieron (representando al 2.4 % del total de empleados que no abandonaron), y si bien este número es reducido, refleja que el modelo sacrifica la detección de empleados en riesgo con tal de no generar falsas alarmas.

Por su parte, incorporar el parámetro `scale_pos_weight` manteniendo el *recall* como métrica de optimización, logra elevar considerablemente la detección de empleados en riesgo hasta un 79 % (37 de 47), no obstante, lo hace a un costo elevado, pues clasifica incorrectamente a 50 empleados, que habrían permanecido. Lo que implicaría destinar recursos de retención en una gran cantidad de empleados que al final no abandonarían.

Si bien la configuración F1 + `scale_pos_weight` mantiene un balance razonable entre recall (0.62) y falsos positivos (26), la configuración final seleccionada es aquella que combina `scale_pos_weight` con F_β ($\beta = 2$), como métrica de optimización, dado que logra identificar correctamente al 74 % de los empleados que abandonaron la compañía (35 de 47), reduciendo los falsos negativos a 12 y manteniendo un nivel de falsos positivos aceptable (35, aproximadamente el 14 % del total de empleados que permanecieron), en congruencia con el supuesto de negocio de que **no detectar a un empleado en riesgo de abandono tiene un costo mayor que intervenir preventivamente sobre uno que habría permanecido**.

Así pues, la combinación óptima identificada por `GridSearchCV` fue la siguiente.

Hiperparámetro	<code>n_estimators</code>	<code>max_depth</code>	<code>learning_rate</code>	<code>min_child_weight</code>
Valor óptimo	100	3	0.05	1

Tabla 6: Hiperparámetros del modelo XGBoost final

Cabe mencionar que en cada configuración, la malla evaluó 54 combinaciones de hiperparámetros, resultando en 270 modelos ajustados en total.

6.4. Perceptrón Multicapa (MLP)

El modelo de Redes Neuronales fue implementado mediante un *Perceptrón Multicapa* (Multilayer Perceptron), siguiendo la metodología propuesta por Géron en *Hands-On Machine Learning with Scikit-Learn, Keras & TensorFlow*, se implementó una red neuronal utilizando la librería o API **Keras** de TensorFlow.

La arquitectura usada, corresponde a una red neuronal secuencial con una estructura en forma de embudo, es decir, conformada con una capa de entrada, dos capas ocultas de 64 y 32 neuronas respectivamente y una capa de salida. En las capas ocultas se utilizó una función de activación *ReLU* la cual transforma los valores negativos en cero y mantiene sin modificación los valores positivos, permitiendo un entrenamiento más eficiente y mitigando el problema del gradiente desvanecido. Así mismo se aplicó un *Dropout* con una tasa de 0.20 como técnica de reularización para reducir el sobreajuste.

La capa de salida está conformada por una única neurona con función de activación **Sigmoid**, que es más apropiada a problemas de clasificación binaria, ya que devuelve la probabilidad comprendida entre 0 y 1 de que un empleado pertenezca a la clase (*Attrition* = 1).

Para el entrenamiento del modelo se utilizó el optimizador **Adam** de Keras, con una tasa de aprendizaje de 0.0005. Fue seleccionado Adam debido a su rápida convergencia y a que se adapta automáticamente a la magnitud de las actualizaciones de los pesos. Como función de pérdida se empleó **Binary Crossentropy** adecuada para problemas de clasificación binaria, ya que penaliza más aquellas predicciones realizadas con alta confianza sobre la clase incorrecta. Con el propósito de reducir el sobreajuste, se implementó la técnica de regularización **Early Stopping**, monitoreando la pérdida sobre el conjunto de validación (*val_loss*). Si dicha métrica no presentaba mejoras durante 15 épocas consecutivas, el entrenamiento se detiene y se restauran los pesos correspondientes al modelo con menor pérdida de validación.

Durante el entrenamiento se emplearon pesos de clase (*class_weight*) calculados mediante la estrategia **balanced**, con el fin de compensar el desbalance existente en la variable objetivo y otorgar una mayor importancia a la correcta identificación de los empleados con riesgo de abandono.

Tabla 7: Resultados del modelo MLP

Precisión	Recall	F1	Accuracy	ROC-AUC	TP	FN	FP
0.45	0.70	0.55	0.81	0.84	33	14	41

La matriz de confusión obtenida fue:

$$\begin{bmatrix} 206 & 41 \\ 14 & 33 \end{bmatrix}$$

Los resultados muestran que el modelo logró identificar correctamente 33 de los 47 empleados que abandonaron la organización, alcanzando un **Recall de 70 %**, métrica priorizada en este proyecto. Si bien la **Precisión fue de 45 %**, el modelo consiguió un equilibrio razonable entre la detección de empleados en riesgo y la cantidad de falsas alarmas generadas. Asimismo, el valor de **ROC-AUC = 0.84** indica una buena capacidad discriminatoria para diferenciar entre empleados que permanecerán y aquellos con riesgo de abandonar la empresa.

7. Evaluación Multimodelo: Rendimiento y Explicabilidad

7.1. Contraste de Rendimiento Predictivo

Dado que el objetivo principal del ejercicio consiste en identificar oportunamente a los empleados con riesgo de abandono, el siguiente cuadro comparativo, se centra en las métricas correspondientes a la clase positiva (*Attrition* = **Yes**), complementadas con el número de verdaderos positivos (TP), falsos negativos (FN) y falsos positivos (FP), de los modelos evaluados sobre el mismo conjunto de prueba ($n = 294$) y un umbral de clasificación (0.5).

Permitiéndonos así evaluar el comportamiento conjunto entre la capacidad de detección de los modelos y su costo asociado a la generación de falsas alarmas.

Modelo	Precisión (1)	Recall (1)	F1-score (1)	TP	FN	FP
Regresión Logística (Lasso)	0.36	0.74	0.49	35	12	62
Regresión Logística (Ridge)	0.36	0.74	0.49	35	12	62
Random Forest	0.45	0.77	0.57	36	11	44
AdaBoost	0.49	0.57	0.53	27	20	28
XGBoost	0.50	0.74	0.60	35	12	35
MLP	0.45	0.70	0.55	33	14	41

Tabla 8: Cuadro comparativo de rendimiento multimodelo sobre el conjunto de prueba.

7.2. Análisis de la Explicabilidad: Lasso vs. XGBoost

Mientras que la importancia de características en *XGBoost* mide la contribución relativa de cada variable en la partición y pureza de los nodos (magnitud absoluta no negativa), los coeficientes de Lasso (L_1) aportan tanto la magnitud como la dirección del efecto (signo del coeficiente).

Las 15 variables más relevantes según el criterio de *XGBoost* se presentan consolidadas y contrastadas en la Tabla 9.

Tabla 9: Contraste de explicabilidad: Top 15 variables ordenadas por importancia en XGBoost frente a los coeficientes optimizados de Lasso (L_1).

Variable	Coefficiente Lasso (L_1)	Importancia XGBoost
JobLevel	-0.0459	0.0857
OverTime	+0.7242	0.0639
StockOptionLevel	-0.2211	0.0580
YearsAtCompany	+0.4635	0.0540
JobRole_Sales Executive	0.0000	0.0519
YearsWithCurrManager	-0.4808	0.0490
Age	-0.1473	0.0363
JobSatisfaction	-0.4011	0.0311
RelationshipSatisfaction	-0.2320	0.0308
MonthlyIncome	-0.2845	0.0281
JobRole_Sales Representative	+0.5837	0.0277
NumCompaniesWorked	+0.3335	0.0261
EnvironmentSatisfaction	-0.3851	0.0254
Department_Sales	0.0000	0.0253
YearsSinceLastPromotion	+0.5517	0.0252

El análisis conjunto de la Tabla 9 revela patrones críticos en el comportamiento de las penalizaciones y la estructura matemática del problema de rotación laboral:

- **Selección Intrínseca (Valores Cero en Lasso):** Mientras que *XGBoost* asigna alta importancia a *JobRole_Sales Executive* (0.0519) y *Department_Sales* (0.0253), Lasso las reduce exactamente a 0.0000. Esto ocurre por el efecto de la norma L_1 ante la multicolinealidad: la información de estas variables ya se encontraba absorbida linealmente por el puesto *JobRole_Sales Representative* (+0.5837) y el nivel de ingresos. Lasso detecta esta redundancia y expulsa los atributos sobrantes para simplificar el modelo.
- **Horas Extra (OverTime):** Ambos modelos coinciden plenamente. Es la segunda variable más relevante para *XGBoost* (0.0639) y tiene el coeficiente positivo más alto en Lasso (+0.7242). Esto confirma que trabajar horas extra es el detonante principal para que un empleado decida renunciar.
- **Impacto del Salario y el Estatus:** Variables como nivel del puesto (*JobLevel*: 0.0459), salario (*MonthlyIncome*: -0.2845) y opciones de acciones (*StockOptionLevel*: -0.2211) son clave en *XGBoost*. Al tener coeficientes negativos en Lasso, se demuestra que una mejor compensación y mayor jerarquía ayudan directamente a retener al personal.
- **Antigüedad y Clima Laboral:** Los años en la empresa (*YearsAtCompany*: +0.4635) y el tiempo desde el último ascenso (*YearsSinceLastPromotion*: +0.5517) aumentan el riesgo de salida. Sin embargo, tener una buena relación con el jefe actual (*YearsWithCurrManager*: -0.4808) es el

retenedor más efectivo, incluso por encima del sueldo. Asimismo, las encuestas de satisfacción con el entorno (**EnvironmentSatisfaction**: -0.3851) y del puesto (**JobSatisfaction**: -0.4011) confirman que tener un buen clima laboral es clave para retener a sus trabajadores.

8. Conclusiones

Si bien *Random Forest* presenta el recall más alto para la clase positiva (0.77), su precisión es apenas de 45%; es decir, de los 80 empleados que predice que se irán, sólo acierta en 36, de modo que más de la mitad de las alertas generadas por el modelo corresponden a falsos positivos.

Ahora bien, aunque los modelos de *Regresión Logística (Lasso y Ridge)* alcanzan el mismo recall que *XGBoost*, la precisión de ambos es considerablemente menor (de 0.36 con 62 falsos positivos), lo que se traduciría en destinar recursos de retención a un número significativamente mayor de empleados que habría permanecido en la compañía.

Por otra parte, *MLP* y *AdaBoost*, presentan desempeños intermedios con recall de 0.70 y 0.57 respectivamente, pero no ofrecen ventajas claras frente a *XGBoost* en ninguna de las métricas reportadas.

Así, dado que *XGBoost* logra identificar correctamente al 74% de los empleados en riesgo de abandono (35 de 47), con 12 falsos negativos y 35 falsos positivos, concluimos que resulta el modelo más adecuado para el objetivo planteado, considerando de manera conjunta el recall, la precisión y el F1 score sobre la clase positiva; donde el balance entre sensibilidad y precisión es resultado de una estrategia de modelado explícitamente orientada al problema: la incorporación de **scale_pos_weight** para compensar el desbalance de clases y el uso de F_β con $\beta = 2$ como métrica de optimización, priorizando la detección de empleados en riesgo sobre la reducción de falsas alarmas, en congruencia con el supuesto de negocio de que **no detectar a un empleado que abandonará la compañía tiene un costo mayor que intervenir preventivamente sobre uno que habría permanecido**.